

Manual de Usuario

Directivas Comunes en ARM

1. **.global**
 - **Función:** Define una etiqueta como global para que sea accesible desde otros archivos de código.
 - **Ejemplo:** `.global main` hace que la etiqueta `main` sea accesible globalmente.
2. **.text**
 - **Función:** Indica el inicio de la sección de código ejecutable.
 - **Ejemplo:** `.text` se utiliza para definir la sección donde comienza el código.
3. **.data**
 - **Función:** Indica el inicio de la sección de datos, donde se definen variables o constantes.
 - **Ejemplo:** `.data` se utiliza para declarar la sección de datos.
4. **.word**
 - **Función:** Reserva espacio en memoria para una palabra (normalmente 32 bits).
 - **Ejemplo:** `.word 0x12345678` almacena el valor hexadecimal en memoria.
5. **.align**
 - **Función:** Alinea la siguiente instrucción o dato a una dirección específica en memoria.
 - **Ejemplo:** `.align 4` alinea los datos a un límite de 4 bytes.

Instrucciones comunes en ARM

1. **MOV (Move)**
 - **Función:** Copia un valor en un registro.
 - **Formato:** `MOV destino, fuente`
 - **Ejemplo:** `MOV R0, #5` mueve el valor inmediato 5 al registro R0.
2. **LDR (Load Register)**
 - **Función:** Carga un valor de memoria en un registro.
 - **Formato:** `LDR destino, [fuente]`
 - **Ejemplo:** `LDR R0, [R1]` carga el valor en la dirección de memoria apuntada por R1 en R0.
3. **STR (Store Register)**
 - **Función:** Almacena el valor de un registro en una dirección de memoria.
 - **Formato:** `STR fuente, [destino]`

- **Ejemplo:** `STR R0, [R1]` almacena el valor de R0 en la dirección de memoria apuntada por R1.
4. **ADD (Add)**
- **Función:** Suma dos valores y almacena el resultado en un registro.
 - **Formato:** `ADD destino, operando1, operando2`
 - **Ejemplo:** `ADD R0, R1, R2` suma los valores de R1 y R2 y almacena el resultado en R0.
5. **SUB (Subtract)**
- **Función:** Resta un valor del otro y almacena el resultado en un registro.
 - **Formato:** `SUB destino, operando1, operando2`
 - **Ejemplo:** `SUB R0, R1, R2` resta el valor de R2 del valor de R1 y almacena el resultado en R0.
6. **CMP (Compare)**
- **Función:** Compara dos valores restándolos para establecer las banderas de condición.
 - **Formato:** `CMP operando1, operando2`
 - **Ejemplo:** `CMP R0, R1` compara los valores en R0 y R1 y actualiza las banderas.
7. **B (Branch)**
- **Función:** Salta a una dirección especificada incondicionalmente.
 - **Formato:** `B etiqueta`
 - **Ejemplo:** `B bucle` salta a la etiqueta `bucle`.
8. **BL (Branch with Link)**
- **Función:** Salta a una dirección especificada y guarda la dirección de retorno en el registro de enlace (LR).
 - **Formato:** `BL etiqueta`
 - **Ejemplo:** `BL subrutina` llama a la subrutina y guarda la dirección de retorno.
9. **BX (Branch and Exchange)**
- **Función:** Cambia el flujo de ejecución al valor en el registro especificado y cambia al modo Thumb si es necesario.
 - **Formato:** `BX registro`
 - **Ejemplo:** `BX LR` cambia el flujo de control al valor almacenado en el registro de enlace.
10. **ADR (Load Address)**
- **Función:** Carga una dirección en un registro.
 - **Formato:** `ADR destino, etiqueta`

- **Ejemplo:** `ADR R0, etiqueta` carga la dirección de `etiqueta` en R0.

11. NOP (No Operation)

- **Función:** Instrucción que no realiza ninguna operación, se utiliza para retrasos o alineación.
- **Formato:** `NOP`

12. BR (Branch to Register)

- **Función:** Cambia el flujo de ejecución al valor en un registro.
- **Formato:** `BR registro`
- **Ejemplo:** `BR X0` cambia el flujo de control a la dirección almacenada en el registro X0.

13. SVC (Supervisor Call)

- **Función:** Genera una llamada al sistema.
- **Formato:** `SVC número`
- **Ejemplo:** `SVC 0` invoca una llamada al sistema con el código 0.

14. STP (Store Pair)

- **Función:** Almacena dos registros consecutivos en memoria.
- **Formato:** `STP reg1, reg2, [dirección]`
- **Ejemplo:** `STP X0, X1, [SP, #16]` almacena los registros X0 y X1 en la pila con un desplazamiento.

15. LDP (Load Pair)

- **Función:** Carga dos registros consecutivos desde la memoria.
- **Formato:** `LDP reg1, reg2, [dirección]`
- **Ejemplo:** `LDP X0, X1, [SP, #16]` carga los registros X0 y X1 desde la pila con un desplazamiento.

16. RET (Return)

- **Función:** Retorna desde una subrutina.
- **Formato:** `RET`
- **Ejemplo:** `RET` retorna al llamador de la subrutina.

17. CBZ (Compare and Branch on Zero)

- **Función:** Compara un registro con cero y salta si es igual a cero.
- **Formato:** `CBZ registro, etiqueta`
- **Ejemplo:** `CBZ R0, salida` salta a `salida` si R0 es cero.

18. CBNZ (Compare and Branch on Non-Zero)

- **Función:** Compara un registro con cero y salta si no es igual a cero.
- **Formato:** `CBNZ registro, etiqueta`
- **Ejemplo:** `CBNZ R0, bucle` salta a `bucle` si R0 no es cero.

19. TBNZ (Test Bit and Branch if Not Zero)

- **Función:** Prueba un bit específico de un registro y salta si no es cero.
- **Formato:** `TBNZ registro, bit, etiqueta`
- **Ejemplo:** `TBNZ R0, #1, salida` salta a `salida` si el bit 1 de R0 no es cero.

20. TBZ (Test Bit and Branch if Zero)

- **Función:** Prueba un bit específico de un registro y salta si es cero.
- **Formato:** `TBZ registro, bit, etiqueta`
- **Ejemplo:** `TBZ R0, #1, salida` salta a `salida` si el bit 1 de R0 es cero.

21. AND (Logical AND)

- **Función:** Realiza una operación lógica AND entre dos registros.
- **Formato:** `AND destino, operando1, operando2`

22. ORR (Logical OR)

- **Función:** Realiza una operación lógica OR entre dos registros.
- **Formato:** `ORR destino, operando1, operando2`

23. EOR (Exclusive OR)

- **Función:** Realiza una operación lógica XOR entre dos registros.
- **Formato:** `EOR destino, operando1, operando2`

24. ADRP (Add with Relocation)

- **Función:** Carga la dirección de una página en memoria relativa al contador de programa (PC). Se utiliza principalmente para direccionamiento de memoria en aplicaciones de 64 bits.
- **Formato:** `ADRP destino, etiqueta`
- **Ejemplo:** `ADRP X0, etiqueta` carga la dirección de la página donde se encuentra la etiqueta en el registro `X0`.

25. LDUR (Load Register Unprivileged)

- **Función:** Carga un valor de una dirección de memoria en un registro sin utilizar la alineación de la dirección.
- **Formato:** `LDUR destino, [base, offset]`
- **Ejemplo:** `LDUR X0, [X1, #4]` carga el valor de la dirección `X1 + 4` en el registro `X0`.

26. STUR (Store Register Unprivileged)

- **Función:** Almacena el valor de un registro en una dirección de memoria sin utilizar la alineación de la dirección.

- **Formato:** `STUR fuente, [base, offset]`
- **Ejemplo:** `STUR X0, [X1, #4]` almacena el valor del registro `X0` en la dirección `X1 + 4`.

27. MUL (Multiply)

- **Función:** Multiplica el contenido de dos registros y almacena el resultado en un registro de destino.
- **Formato:** `MUL destino, fuente1, fuente2`
- **Ejemplo:** `MUL R0, R1, R2` multiplica los valores de `R1` y `R2`, y almacena el resultado en `R0`.

28. DIV (Divide)

- **Función:** Realiza la división entera entre dos registros, almacenando el cociente en el registro de destino.
- **Formato:** `DIV destino, dividendo, divisor`
- **Ejemplo:** `DIV R0, R1, R2` divide el valor en `R1` por el valor en `R2` y almacena el resultado en `R0`.

29. MVN (Move Not)

- **Función:** Realiza un complemento a nivel de bits del valor de un registro y lo almacena en el registro de destino.
- **Formato:** `MVN destino, fuente`
- **Ejemplo:** `MVN R0, R1` invierte todos los bits del registro `R1` y almacena el resultado en `R0`.

30. TST (Test)

- **Función:** Realiza una operación AND a nivel de bits entre dos registros y actualiza las banderas de condición sin almacenar el resultado.
- **Formato:** `TST registro1, registro2`
- **Ejemplo:** `TST R0, R1` realiza la operación `R0 AND R1` y actualiza las banderas de condición.

31. CMN (Compare Negative)

- **Función:** Suma el valor de un registro con el valor negativo de otro y actualiza las banderas de condición.

- **Formato:** `CMN registro1, registro2`
- **Ejemplo:** `CMN R0, R1` realiza la comparación $R0 + (-R1)$ y actualiza las banderas de condición.

32. LDRB (Load Register Byte)

- **Función:** Carga un byte desde una dirección de memoria en un registro.
- **Formato:** `LDRB destino, [dirección]`
- **Ejemplo:** `LDRB R0, [R1]` carga el byte en la dirección apuntada por `R1` en `R0`.

33. UXTB (Unsigned Extend Byte)

- **Función:** Extiende un byte sin signo a un valor de 32 o 64 bits.
- **Formato:** `UXTB destino, fuente`
- **Ejemplo:** `UXTB R0, R1` extiende el byte menos significativo de `R1` a un valor de 32 bits en `R0`.

34. STRB (Store Register Byte)

- **Función:** Almacena un byte del registro en una dirección de memoria.
- **Formato:** `STRB fuente, [dirección]`
- **Ejemplo:** `STRB R0, [R1]` almacena el byte menos significativo de `R0` en la dirección apuntada por `R1`.

35. SDIV (Signed Divide)

- **Función:** Realiza la división entera de dos registros con signo.
- **Formato:** `SDIV destino, dividendo, divisor`
- **Ejemplo:** `SDIV R0, R1, R2` divide el valor en `R1` por el valor en `R2` y almacena el resultado en `R0`.

36. BLT (Branch if Less Than)

- **Función:** Salta a una etiqueta si el resultado anterior fue menor que cero (condición de bandera N).
- **Formato:** `BLT etiqueta`
- **Ejemplo:** `BLT etiqueta1` salta a `etiqueta1` si la comparación anterior fue menor que cero.

37. BEQ (Branch if Equal)

- **Función:** Salta a una etiqueta si la comparación anterior fue igual (condición de bandera Z).
- **Formato:** BEQ etiqueta
- **Ejemplo:** BEQ etiqueta2 salta a etiqueta2 si el resultado anterior fue igual a cero.

38. BLE (Branch if Less or Equal)

- **Función:** Salta a una etiqueta si el resultado anterior fue menor o igual que cero (condición de bandera N o Z).
- **Formato:** BLE etiqueta
- **Ejemplo:** BLE etiqueta3 salta a etiqueta3 si el resultado anterior fue menor o igual a cero.

39. BNE (Branch if Not Equal)

- **Función:** Salta a una etiqueta si la comparación anterior no fue igual (condición de bandera no Z).
- **Formato:** BNE etiqueta
- **Ejemplo:** BNE etiqueta4 salta a etiqueta4 si el resultado anterior no fue igual a cero.

40. BGT (Branch if Greater Than)

- **Función:** Salta a una etiqueta si el resultado anterior fue mayor que cero (condición de bandera N).
- **Formato:** BGT etiqueta
- **Ejemplo:** BGT etiqueta5 salta a etiqueta5 si el resultado anterior fue mayor que cero.